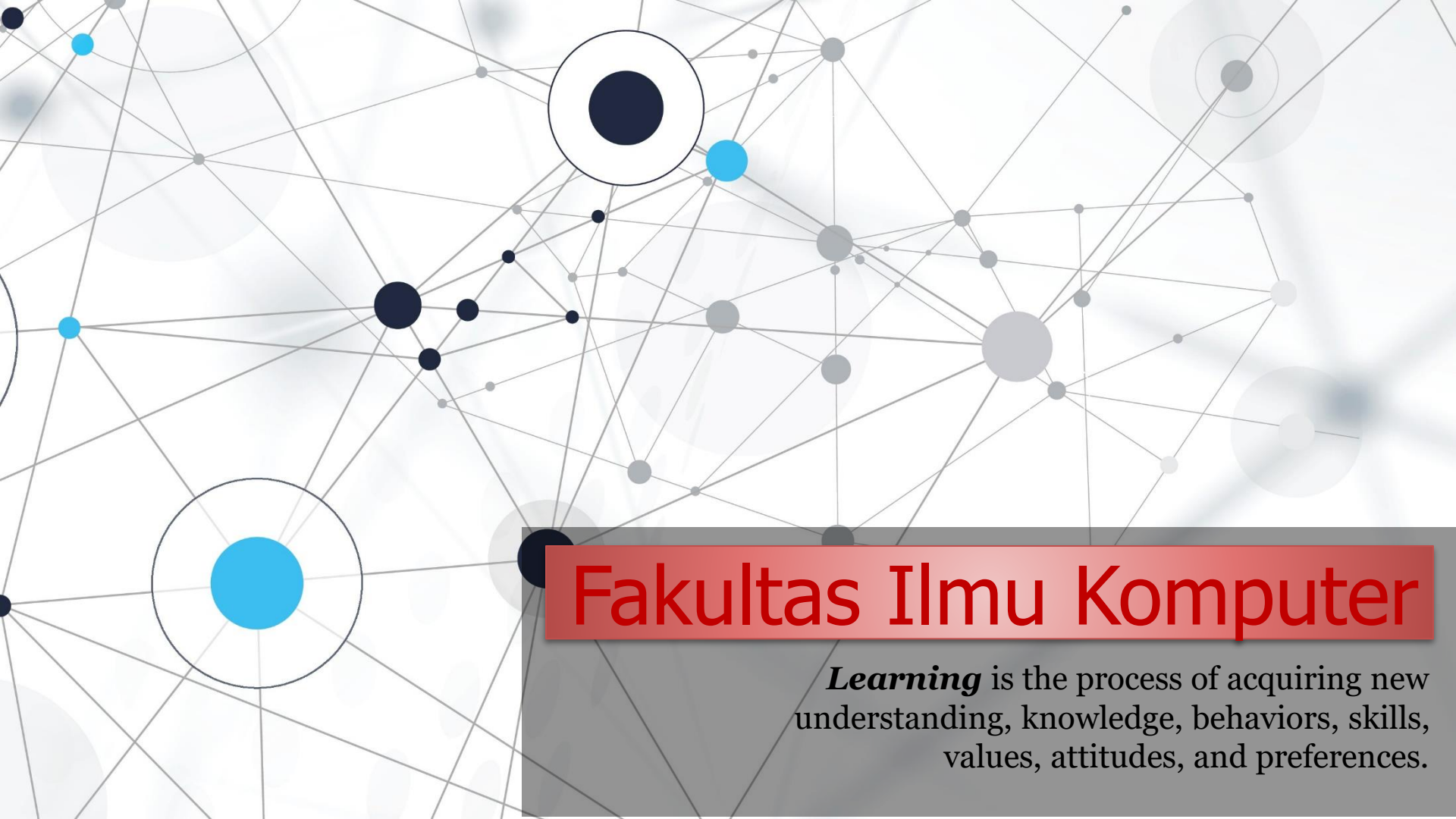


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

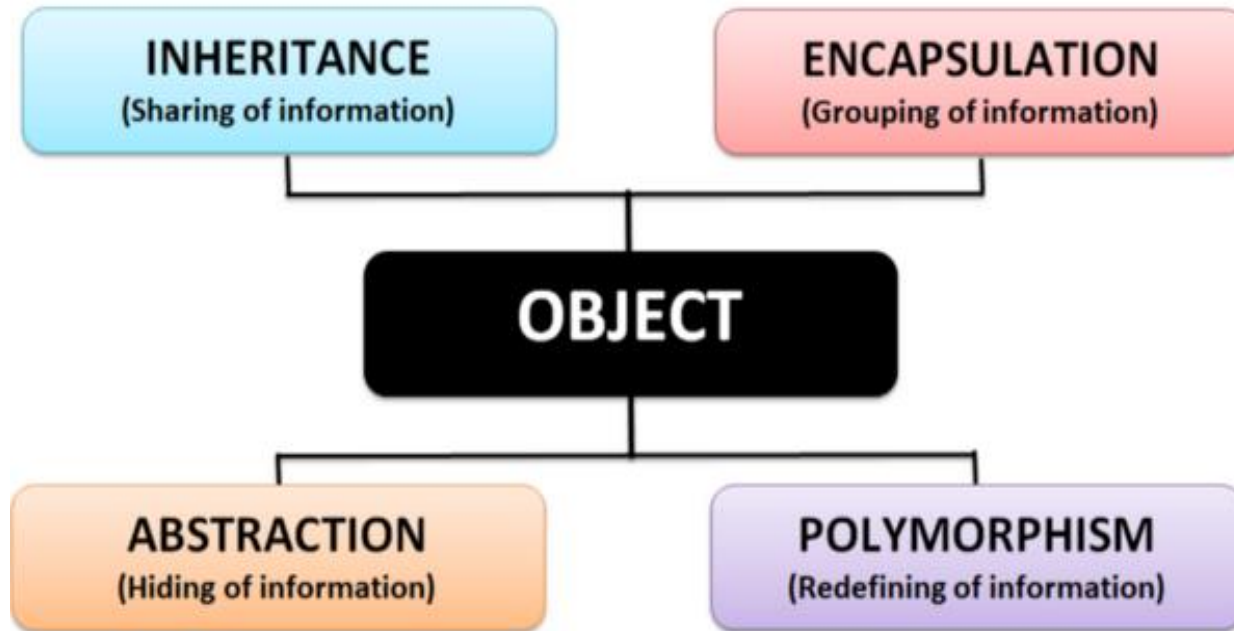
Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

OOP Python - Method Overriding



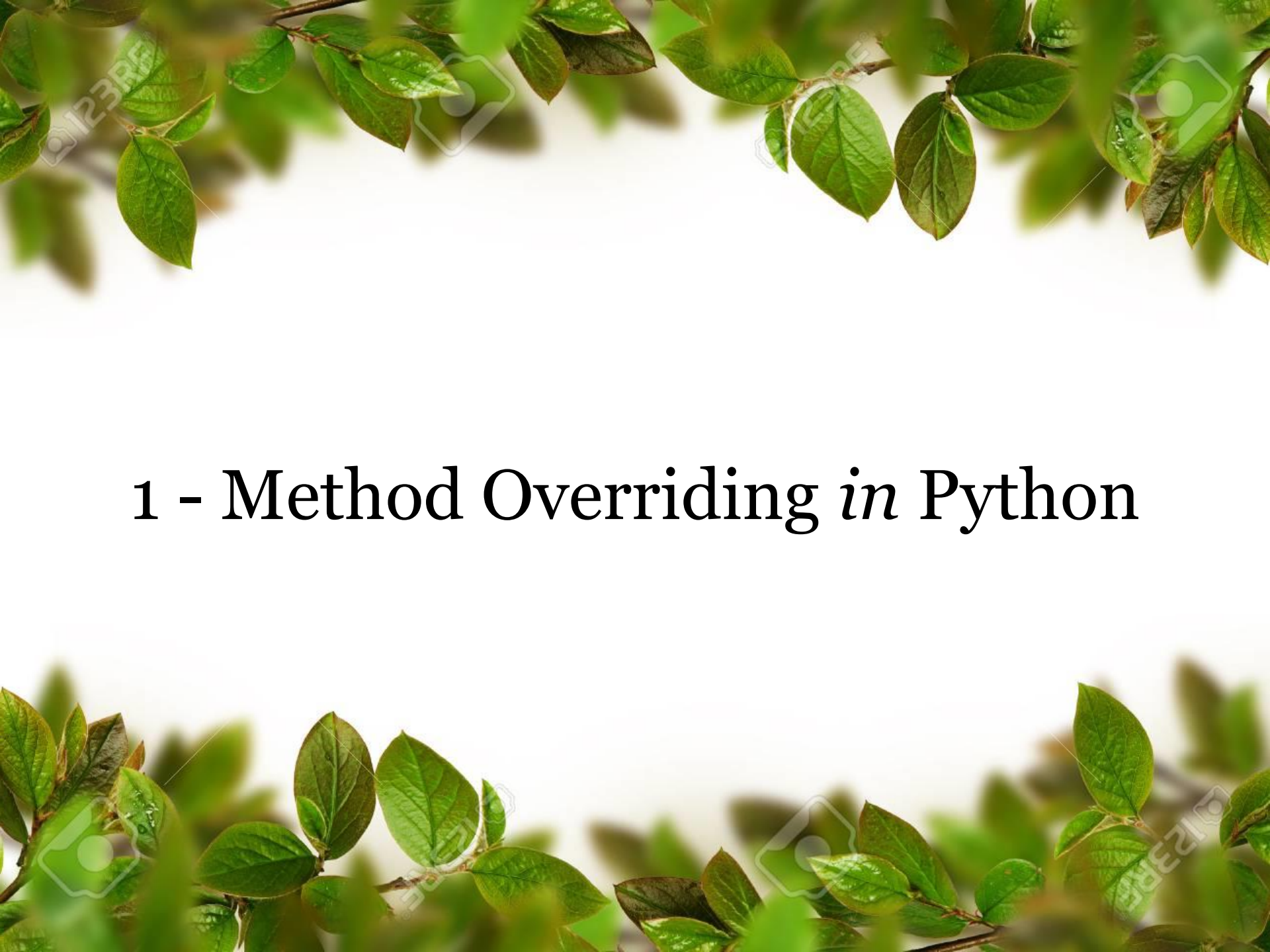
**Welcome to *the* Object Oriented
Programming *course***

Understanding *the* Concept of OOP



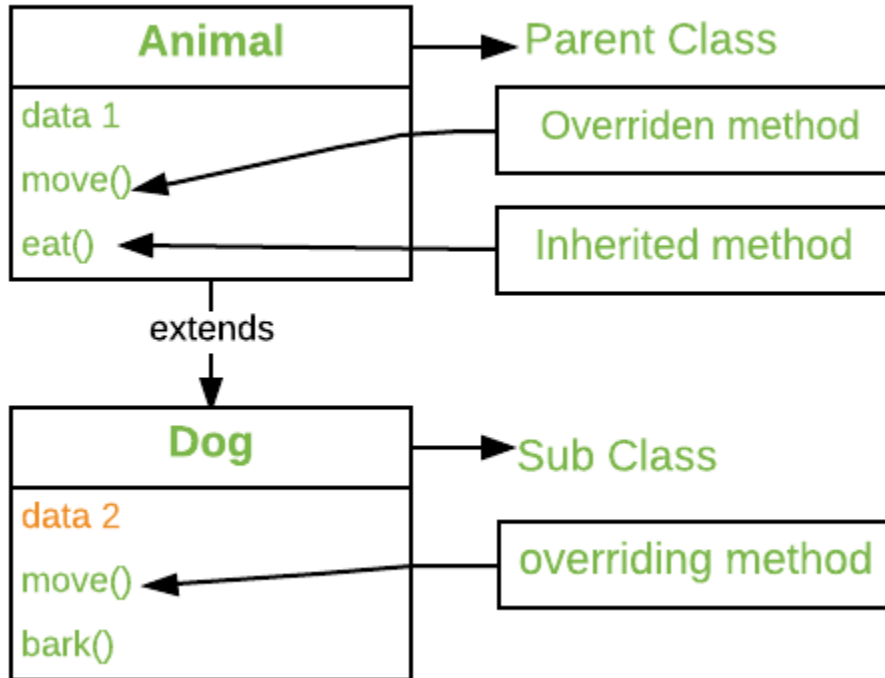
- ✓ Object Oriented Programming (OOP) atau pemrograman berorientasi objek adalah salah satu pendekatan paling efektif dalam menulis kode suatu program/software.
- ✓ Pada OOP, kita menulis class yang merepresentasikan suatu situasi seperti pada keadaan sebenarnya dan kita dapat membuat suatu objek berdasarkan class tersebut.

Inheritance provides code reusability to the program because we can use an existing class to create a new class instead of creating it from scratch.



1 - Method Overriding *in* Python

What is Method Overriding ???

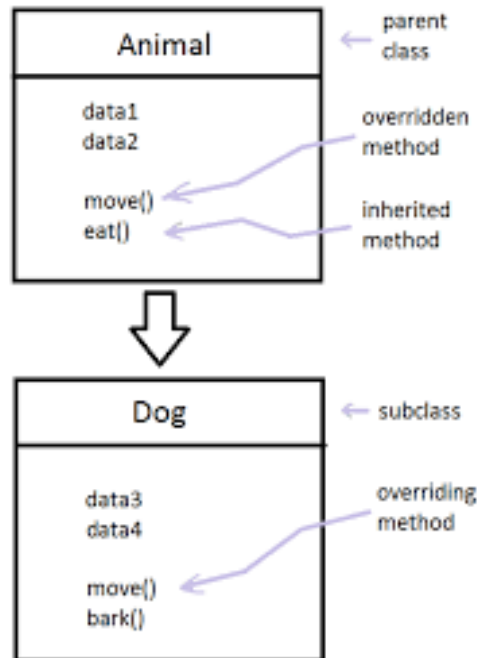


Method overriding is an **ability** of any object-oriented programming language that **allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes.**

When a method in a subclass has the **same name, same parameters** or signature and **same return type (or sub-type)** as a method in its super-class, then the method in the subclass is called to override the method in the super-class.

Method overriding juga dikenal sebagai substitusi, penimpaan atau penggantian metode.

What is Method Overriding ???



- ❖ **Method overriding** adalah konsep dalam pemrograman berorientasi objek (OOP) **di mana sebuah subclass dapat mengganti implementasi suatu metode yang telah didefinisikan di superclassnya**. Subclass dapat memodifikasi perilaku metode yang diwarisi dari superclassnya.
- ❖ Dalam Python, method overriding dilakukan dengan **mendefinisikan kembali method parent class di dalam child class** dengan **nama yang sama**.
- ❖ Ketika method tersebut dipanggil pada object child class, **implementasi di dalam child class yang akan dieksekusi**, bukan implementasi dari parent class.

Manfaat Method Overring *di* OOP

The use of **method overriding in OOP** is to make it possible for changing the behavior of a method in a subclass inherited from a parent class (*superclass*). Here are some of the benefits of using method overriding in OOP:

Polymorphism (ability to take different forms)	<u>Method overriding</u> allows objects of different classes to respond in different ways to the same method call. In this case, method overriding can be used to apply polymorphism to the program.
Flexibility	<u>Method overriding</u> allows a child class (subclass) to extend or improve the behavior of methods inherited from a parent class (superclass) without changing the behavior of the parent class itself. In this way, the use of method overriding allows flexibility in program development.
Increased efficiency	By using <u>method overriding</u> , programmers can optimize program performance by applying code that matches the type of object received as input to the overridden method.
Code reduction	By using <u>method overriding</u> , programmers can avoid rewriting the same code in a subclass that inherits from the parent class, thus reducing the amount of code that must be written.

Kekurangan Method Overring *di* OOP

Meskipun **method overriding** pada OOP memiliki banyak manfaat, penggunaannya juga memiliki beberapa kekurangan, di antaranya:

1) Errors in code:

In this case, we must be careful if the programmer does not fully understand how method overriding works, then there can be errors in its use. Such errors can produce undesirable results in the program.

2) Complexity:

The use of method overriding can add complexity to the program as it can cause dependency between the child class (subclass) and the parent class (superclass).

3) Confusion:

Sometimes, programmers can fall into the trap of programming with method overriding which makes the code difficult to read and understand. This can lead to difficulties in fixing and maintaining the code in the future.

4) Limitations in development:

The use of method overriding can limit future development of the program. If too many subclasses override a superclass, it can make it difficult to make changes to the superclass.

5) Dependency on parent class:

If the parent class (superclass) changes, then the child classes (subclasses) that override the methods of the parent class must also be adjusted. This can lead to excessive dependency on the parent class and hinder flexibility in program development.

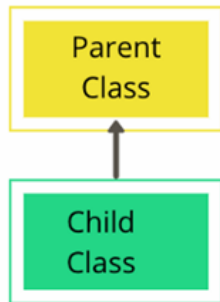
A decorative border of green leaves and branches at the top of the slide.

2 - Method Overriding Bergantung Pada Proses Inheritance

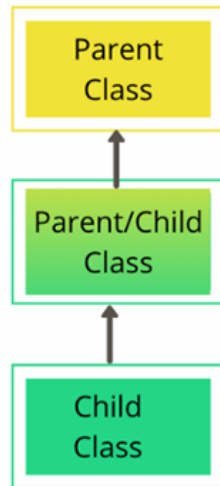
A decorative border of green leaves and branches at the bottom of the slide.

Method Overriding *with* Inheritance

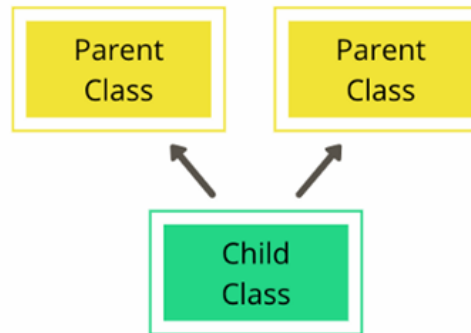
Simple(Single)
Inheritance



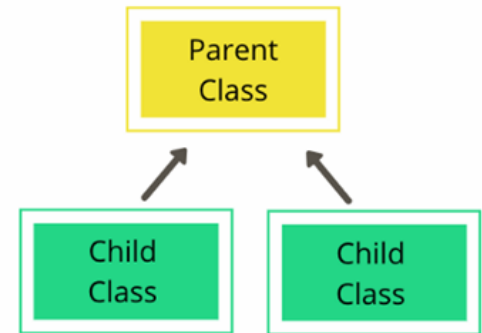
Multi Level
Inheritance



Multiple
Inheritance



Hierarchical
Inheritance



- ❖ **Method overriding** is a technique where **a method defined in a superclass or parent class is also redefined in a subclass**. In this sense, the subclass has a method with the same name as the method in the parent class, and implements a different behavior.
- ❖ **Method overriding** in OOP relies on the process of inheritance. Inheritance in OOP allows a subclass to inherit properties or behaviors from a parent class (superclass) and can change those behaviors through method overriding.

Method Overriding *with* Single Inheritance

```
1 # Defining parent class
2 class Parent():
3
4     # Constructor
5     def __init__(self):
6         self.value = "Inside Parent"
7
8     # Parent's show method
9     def show(self):
10        print(self.value)
11
12 # Defining child class
13 class Child(Parent):
14
15     # Constructor
16     def __init__(self):
17         self.value = "Inside Child"
18
19     # Child's show method
20     def show(self):
21        print(self.value)
22
23
24 # create objects of the class
25 obj1 = Parent()
26 obj2 = Child()
27
28 obj1.show()
29 obj2.show()
```

Method overriding pada Single Inheritance mengacu pada penggunaan inheritance dengan hanya satu kelas anak (*subclass*) yang mewarisi sifat atau perilaku dari satu kelas induk (*superclass*).

Dalam Single Inheritance, ketika sebuah method pada kelas anak (*subclass*) memiliki nama yang sama dengan method pada kelas induk (*superclass*), maka method pada kelas anak (*subclass*) **akan meng-overwrite (mengganti) method pada kelas induk (*superclass*) dengan perilaku yang baru.**

Method Overriding *with* Multiple Inheritance

```
1 # Defining parent class 1
2 class Parent1():
3
4     # Parent's show method
5     def show(self):
6         print("Inside Parent1")
7
8 # Defining Parent class 2
9 class Parent2():
10
11     # Parent's show method
12     def display(self):
13         print("Inside Parent2")
14
15
16 # Defining child class
17 class Child(Parent1, Parent2):
18
19     # Child's show method
20     def show(self):
21         print("Inside Child")
22
23
24 # create objects of the class
25 obj = Child()
26
27 obj.show()
28 obj.display()
```

Method overriding pada Multiple Inheritance adalah proses penggunaan inheritance dengan dua atau lebih kelas anak (*subclass*) yang mewarisi sifat atau perilaku dari dua atau lebih kelas induk (*superclass*).

Dalam Multiple Inheritance, jika terdapat method pada dua kelas induk (*superclass*) yang sama nama dan parameter, maka ketika method tersebut dipanggil pada kelas anak (*subclass*), maka method yang paling dekat dengan kelas anak (*subclass*) yang akan dipanggil.

Remember !!!

Jika kedua kelas induk (*superclass*) memiliki method dengan NAMA YANG SAMA dan PARAMETER yang berbeda, maka tidak terjadi overriding, dan keduanya akan diwarisi oleh kelas anak (*subclass*). APAKAH BENAR?

Method Overriding *with* Multilevel Inheritance

```
1 # Defining parent class
2 class Parent():
3
4     # Parent's show method
5     def display(self):
6         print("Inside Parent")
7
8
9 # Inherited or Sub class
10 class Child(Parent):
11
12     # Child's show method
13     def show(self):
14         print("Inside Child")
15
16 # Inherited or Sub class
17 class GrandChild(Child):
18
19     # Child's show method
20     def show(self):
21         print("Inside GrandChild")
22
23 # Create objects of the class
24 g = GrandChild()
25 g.show()
26 g.display()
```

Method Overriding pada Multilevel Inheritance mengacu pada penggunaan inheritance dengan tiga atau lebih kelas dengan hierarki yang terbentuk dalam rangkaian pewarisan.

Dalam Multilevel Inheritance, ketika kelas anak (*subclass*) mewarisi sifat atau perilaku dari kelas induk (*superclass*) dan kemudian kelas induk (*superclass*) tersebut mewarisi sifat atau perilaku dari kelas induk lainnya, maka ketika method yang sama diwariskan oleh kedua kelas, maka method yang paling dekat dengan kelas anak (*subclass*) yang akan dipanggil.



Game *for* Students
(Menebak output Python program)

Student Game #1

```
game1.py X
1 # Parent #1
2 class Parent1():
3     def show(self):
4         print("Parent One !!!");
5
6 # Child Class
7 class Child(Parent1):
8     def show(self):
9         print("This is child class.");
10
11 # create object from class
12 obj_child = Child();
13 obj_child.show();
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**

```
1 # Parent #1
2 class Parent1():
3     def show(self):
4         print("Parent One !!!");
5
6 # Child Class
7 class Child(Parent1):
8     def show(self):
9         print("This is child class.");
10
11 # create object from class
12 obj_child = Child();
13 obj_child.show();
14
15
```

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game1.py',
wdir='D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')
This is child class.

In [2]:

Student Game #2

```
game2.py X
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self):
9          print("Parent Two.");
10
11 # Child Class
12 class Child(Parent1, Parent2):
13     def test(self):
14         print("This is child class.");
15
16 # create object from class
17 obj_child = Child();
18
19 # call method from object
20 obj_child.show();
21
```

Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!



```
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self):
9          print("Parent Two.");
10
11 # Child Class
12 class Child(Parent1, Parent2):
13     def test(self):
14         print("This is child class.");
15
16 # create object from class
17 obj_child = Child();
18
19 # call method from object
20 obj_child.show();
21
22
```

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game2.py',
wdir='D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')
Parent One.

In [2]: |

Student Game #3

```
game3.py X
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self):
9          print("Parent Two.");
10
11 # Child Class
12 class Child(Parent2, Parent1):
13     def test(self):
14         print("This is child class.");
15
16 # create object from class
17 obj_child = Child();
18
19 # call method from object
20 obj_child.show();
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**



```
1 # Parent #1
2 class Parent1():
3     def show(self):
4         print("Parent One.");
5
6 # Parent #2
7 class Parent2():
8     def show(self):
9         print("Parent Two.");
10
11 # Child Class
12 class Child(Parent2, Parent1):
13     def test(self):
14         print("This is child class.");
15
16 # create object from class
17 obj_child = Child();
18
19 # call method from object
20 obj_child.show();
21
22
```

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game3.py',
wdir='D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')
Parent Two.

In [2]:

Student Game #4

```
game4.py X
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self):
9          print("Parent Two.");
10
11 # Child Class
12 class Child(Parent1, Parent2):
13     def test(self):
14         print("This is child class.");
15
16     def show(self, name="Semmy Taju"):
17         print("My name is {0}.".format(name));
18
19 # create object from class
20 obj_child = Child();
21
22 # call method from object
23 obj_child.show();
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**

D:\UNCLAB CLASSES\CLASS\Object-Oriented Programming\CODES\Lecture17 [overriding]\Game\game4.py

game4.py X

```
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self):
9          print("Parent Two.");
10
11 # Child Class
12 class Child(Parent1, Parent2):
13     def test(self):
14         print("This is child class.");
15
16     def show(self, name="Semmy Taju"):
17         print("My name is {0}.".format(name));
18
19 # create object from class
20 obj_child = Child();
21
22 # call method from object
23 obj_child.show();
24
25
```

Console 6/A X

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game4.py',
wdir='D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')
My name is Semmy Taju.

In [2]:

Student Game #5

```
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self, name="Putri Taju"):
9          print("My name is {0}.".format(name));
10
11
12 # Child Class
13 class Child(Parent1, Parent2):
14     def test(self):
15         print("This is child class.");
16
17     def show(self, name="Semmy Taju"):
18         print("My name is {0}.".format(name));
19
20 # create object from class
21 obj_child = Child();
22
23 # call method from object
24 obj_child.show("Juan Bagadang");
25
```

Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!



game5.py X

```
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self, name="Putri Taju"):
9          print("My name is {0}.".format(name));
10
11
12 # Child Class
13 class Child(Parent1, Parent2):
14     def test(self):
15         print("This is child class.");
16
17     def show(self, name="Semmy Taju"):
18         print("My name is {0}.".format(name));
19
20 # create object from class
21 obj_child = Child();
22
23 # call method from object
24 obj_child.show("Juan Bagadang");
25
26
```

Console 7/A X

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

```
In [1]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented
Programming/CODES/Lecture17 [overriding]/Game/game5.py',
wdir='D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/
CODES/Lecture17 [overriding]/Game')
My name is Juan Bagadang.
```

```
In [2]:
```

Student Game #6

```
game6.py X
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def show(self):
9          print("Parent Two.");
10
11 # Parent #3
12 class Parent3():
13     def show(self):
14         print("Parent Three.");
15
16 # Child Class
17 class Child(Parent3, Parent2, Parent1):
18     def evaluate(self):
19         print("This is child class.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.show();
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**



D:\UNCLAB CLASSES\CLASS\Object-Oriented Programming\CODES\Lecture17 [overriding]\Game\game6.py

game6.py X

```
1 # Parent #1
2 class Parent1():
3     def show(self):
4         print("Parent One.");
5
6 # Parent #2
7 class Parent2():
8     def show(self):
9         print("Parent Two.");
10
11 # Parent #3
12 class Parent3():
13     def show(self):
14         print("Parent Three.");
15
16 # Child Class
17 class Child(Parent3, Parent2, Parent1):
18     def evaluate(self):
19         print("This is child class.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.show();
26
27
```

Console 8/A X

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game6.py',
wdir='D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')
Parent Three.

In [2]:

Student Game #7

```
game7.py X
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def shows(self):
9          print("Parent Two.");
10
11 # Parent #3
12 class Parent3():
13     def shows(self):
14         print("Parent Three.");
15
16 # Child Class
17 class Child(Parent3, Parent2, Parent1):
18     def shows(self):
19         print("This is child class.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.show();
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**



game7.py X

```
1  # Parent #1
2  class Parent1():
3      def show(self):
4          print("Parent One.");
5
6  # Parent #2
7  class Parent2():
8      def shows(self):
9          print("Parent Two.");
10
11 # Parent #3
12 class Parent3():
13     def shows(self):
14         print("Parent Three.");
15
16 # Child Class
17 class Child(Parent3, Parent2, Parent1):
18     def shows(self):
19         print("This is child class.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.show();
26
27
```

Console 9/A X

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

```
In [1]: runfile('D:/UNCLAB CLASSES/CLASS/Object-Oriented
Programming/CODES/Lecture17 [overriding]/Game/game7.py',
wdir='D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/
CODES/Lecture17 [overriding]/Game')
Parent One.
```

```
In [2]:
```

Student Game #8

```
game8.py X
1  # Parent #1
2  class Parent1():
3      def skill(self):
4          print("Parent skill One.");
5
6  # Parent #2
7  class Parent2():
8      def skill(self):
9          print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self):
14         print("Parent skill Three.");
15
16 # Child Class
17 class Child(Parent1, Parent2, Parent3):
18     def skills(self):
19         print("My skill is Programming.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.skill();
26
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

D:\UNKLAB CLASSES\CLASS\Object-Oriented Programming\CODES\Lecture17 [overriding]\Game\game8.py

game8.py

```
1 # Parent #1
2 class Parent1():
3     def skill(self):
4         print("Parent skill One.");
5
6 # Parent #2
7 class Parent2():
8     def skill(self):
9         print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self):
14         print("Parent skill Three.");
15
16 # Child Class
17 class Child(Parent1, Parent2, Parent3):
18     def skills(self):
19         print("My skill is Programming.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.skill();
26
27
```

Console 10/A

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game9.py', wdir='D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')
Parent skill One.

In [2]:

LSP Python: ready conda: base (Python 3.9.7) Line 27, Col 1 UTF-8 CRLF RW Mem 87%

Student Game #9

```
game9.py X
1  # Parent #1
2  class Parent1():
3      def skill(self, name):
4          print("Parent skill One.");
5
6  # Parent #2
7  class Parent2():
8      def skill(self, name, age):
9          print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self, name, age, gender):
14         print("Parent skill Three.");
15
16 # Child Class
17 class Child(Parent1, Parent2, Parent3):
18     def skill(self):
19         print("My skill is Programming.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.skill();
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

D:\UNCLAB CLASSES\CLASS\Object-Oriented Programming\CODES\Lecture17 [overriding]\Game\game9.py

game9.py X

```
1 # Parent #1
2 class Parent1():
3     def skill(self, name):
4         print("Parent skill One.");
5
6 # Parent #2
7 class Parent2():
8     def skill(self, name, age):
9         print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self, name, age, gender):
14         print("Parent skill Three.");
15
16 # Child Class
17 class Child(Parent1, Parent2, Parent3):
18     def skill(self):
19         print("My skill is Programming.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.skill();
26
27
```

Console 11/A X

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game9.py', wdir='D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')
My skill is Programming.

In [2]:

LSP Python: ready conda: base (Python 3.9.7) Line 27, Col 1 ASCII CRLF RW Mem 86%

Student Game #10

```
game10.py X
1  # Parent #1
2  class Parent1():
3      def skill(self, name):
4          print("Parent skill One.");
5
6  # Parent #2
7  class Parent2():
8      def skill(self, name, age):
9          print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self, name, age, gender):
14         print("Parent skill Three.");
15
16 # Child Class
17 class Child(Parent1, Parent2, Parent3):
18     def skill(self, job):
19         print("My skill is Programming.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.skill();
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

D:\UNCLAB CLASSES\CLASS\Object-Oriented Programming\CODES\Lecture17 [overriding]\Game\game10.py

game10.py

```
1 # Parent #1
2 class Parent1():
3     def skill(self, name):
4         print("Parent skill One.");
5
6 # Parent #2
7 class Parent2():
8     def skill(self, name, age):
9         print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self, name, age, gender):
14         print("Parent skill Three.");
15
16 # Child Class
17 class Child(Parent1, Parent2, Parent3):
18     def skill(self, job):
19         print("My skill is Programming.");
20
21 # create object from class
22 obj_child = Child();
23
24 # call method from object
25 obj_child.skill();
26
27
```

Console 12/A

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game10.py', wdir='D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')

Traceback (most recent call last):

File "D:/UNCLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game10.py", line 25, in <module>
obj_child.skill();

TypeError: skill() missing 1 required positional argument: 'job'

In [2]:

Run file

LSP Python: ready conda: base (Python 3.9.7) Line 27, Col 1 UTF-8 CRLF RW Mem 85%

Student Game #11

```
game11.py X
1  # Parent #1
2  class Parent1():
3      def skill(self, name):
4          print("Parent skill One.");
5
6  # Parent #2
7  class Parent2():
8      def skill(self, name, age):
9          print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self, name, age, gender):
14         print("Parent skill Three.");
15
16 # Parent #4
17 class Parent4():
18     def skill(self, name, age, gender, job):
19         print("Parent skill Four.");
20
21 # Child Class
22 class Child(Parent1, Parent2, Parent3, Parent4):
23     def skill(self):
24         print("My skill is Programming.");
25
26 # create object from class
27 obj_child = Child();
28
29 # call method from object
30 obj_child.skill("Juan Taju", "27", "Male");
```

**Tebaklah output Python program di atas?
Error juga merupakan bagian dari opsi yang bisa dipilih !!!**

Spyder (Python 3.9)

File Edit Search Source Run Debug Consoles Projects Tools View Help

D:\UNKLAB CLASSES\CLASS\Object-Oriented Programming\CODES\Lecture17 [overriding]\Game\game11.py

game11.py

```
1 # Parent #1
2 class Parent1():
3     def skill(self, name):
4         print("Parent skill One.");
5
6 # Parent #2
7 class Parent2():
8     def skill(self, name, age):
9         print("Parent skill Two.");
10
11 # Parent #3
12 class Parent3():
13     def skill(self, name, age, gender):
14         print("Parent skill Three.");
15
16 # Parent #4
17 class Parent4():
18     def skill(self, name, age, gender, job):
19         print("Parent skill Four.");
20
21 # Child Class
22 class Child(Parent1, Parent2, Parent3, Parent4):
23     def skill(self):
24         print("My skill is Programming.");
25
26 # create object from class
27 obj_child = Child();
28
29 # call method from object
30 obj_child.skill("Juan Taju", "27", "Male");
31
32
```

Console 15/A

Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game11.py', wdir='D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game')

Traceback (most recent call last):

File "D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/CODES/Lecture17 [overriding]/Game/game11.py", line 30, in <module>

obj_child.skill("Juan Taju", "27", "Male");

TypeError: skill() takes 1 positional argument but 4 were given

In [2]:

LSP Python: ready conda: base (Python 3.9.7) Line 11, Col 12 UTF-8 CRLF RW Mem 83%



Exercise *for* Students

Exercise #1

```
1 class Buah:
2     def __init__(self, nama, harga, stok):
3         self.nama = nama
4         self.harga = harga
5         self.stok = stok
6
7     def beli(self, jumlah):
8         if jumlah <= self.stok:
9             self.stok -= jumlah
10            return self.harga * jumlah
11        else:
12            return 0
13
14    def tambah_stok(self, jumlah):
15        self.stok += jumlah
16
17    def get_nama(self):
18        return self.nama
19
20    def get_harga(self):
21        return self.harga
22
23    def get_stok(self):
24        return self.stok
25
26    def get_info(self):
27        return f"{self.nama}, harga: {self.harga}, stok: {self.stok}"
```

```
29 class Customer:
30     def __init__(self, nama, saldo):
31         self.nama = nama
32         self.saldo = saldo
33
34     def beli(self, buah, jumlah):
35         total_harga = buah.beli(jumlah)
36         if total_harga > 0 and self.saldo >= total_harga:
37             self.saldo -= total_harga
38             return f"{self.nama} membeli {jumlah} {buah.get_nama()} dengan harga {total_harga}"
39         elif total_harga == 0:
40             return f"Maaf, {buah.get_nama()} sedang kosong"
41         else:
42             return f"Maaf, saldo Anda tidak mencukupi"
43
44     def topup(self, jumlah):
45         self.saldo += jumlah
46
47     def get_saldo(self):
48         return self.saldo
49
50     def get_info(self):
51         return f"{self.nama}, saldo: {self.saldo}"
52
53     def __str__(self):
54         return self.nama
```

```
56 if __name__ == '__main__':
57     apel = Buah('apel', 5000, 10)
58     mangga = Buah('mangga', 7000, 5)
59     john = Customer('John', 50000)
60
61     print(apel.get_info())
62     print(mangga.get_info())
63
64     print(john.beli(apel, 3))
65     print(john.get_info())
66     print(apel.get_info())
67
68     print(john.beli(mangga, 10))
69     print(john.get_info())
70     print(mangga.get_info())
71
72     john.topup(100000)
73     print(john.get_info())
74
75     print(john.beli(apel, 5))
76     print(john.get_info())
77     print(apel.get_info())
78
79     print(john.beli(mangga, 2))
80     print(john.get_info())
81     print(mangga.get_info())
```

Output Program:

```
apel, harga: 5000, stok: 10
mangga, harga: 7000, stok: 5
John membeli 3 apel dengan harga 15000
John, saldo: 35000
apel, harga: 5000, stok: 7
Maaf, mangga sedang kosong
John, saldo: 35000
mangga, harga: 7000, stok: 5
John, saldo: 135000
John membeli 5 apel dengan harga 25000
John, saldo: 110000
apel, harga: 5000, stok: 2
John membeli 2 mangga dengan harga 14000
John, saldo: 96000
mangga, harga: 7000, stok: 3
```

Apakah ada proses
pewarisan???

**Perhatikan code di exercise ke-1,
tambahkan seorang customer dengan
nama = “Semmy” dan saldo = 10000,
selanjutnya lakukanlah transaksi yang
sama seperti “john” tanpa topup
saldo, apakah terjadi error jika tanpa
topup saldo???**

Exercise #2

```
1 class Shape:
2     def __init__(self, color, x, y):
3         self.color = color
4         self.x = x
5         self.y = y
6
7     def area(self):
8         print("Undefined area")
9
10    def perimeter(self):
11        print("Undefined perimeter")
12
13    def move(self, x, y):
14        self.x += x
15        self.y += y
16
17    def set_color(self, color):
18        self.color = color
19
20    def get_color(self):
21        return self.color
```

```
23 class Rectangle(Shape):
24     def __init__(self, color, x, y, width, height):
25         super().__init__(color, x, y)
26         self.width = width
27         self.height = height
28
29     def area(self):
30         return self.width * self.height
31
32     def perimeter(self):
33         return 2 * (self.width + self.height)
```

```
35 class Circle(Shape):
36     def __init__(self, color, x, y, radius):
37         super().__init__(color, x, y)
38         self.radius = radius
39
40     def area(self):
41         return 3.14 * self.radius * self.radius
42
43     def perimeter(self):
44         return 2 * 3.14 * self.radius
```

```
46 if __name__ == '__main__':
47     rectangle = Rectangle('blue', 0, 0, 5, 3)
48     circle = Circle('red', 0, 0, 5)
49
50     print(f"Rectangle area: {rectangle.area()}")
51     print(f"Rectangle perimeter: {rectangle.perimeter()}")
52
53     print(f"Circle area: {circle.area()}")
54     print(f"Circle perimeter: {circle.perimeter()}")
55
56     rectangle.move(2, 3)
57     print(f"Rectangle position: ({rectangle.x}, {rectangle.y})")
58
59     circle.set_color('green')
60     print(f"Circle color: {circle.get_color()}")
```

Output Program:

```
Rectangle area: 15
Rectangle perimeter: 16
Circle area: 78.5
Circle perimeter: 31.400000000000002
Rectangle position: (2, 3)
Circle color: green
```

```
In [3]:
```

Exercise #3

```
1 class Mobil:
2     def __init__(self, merk, tahun, harga_sewa, status):
3         self.merk = merk
4         self.tahun = tahun
5         self.harga_sewa = harga_sewa
6         self.status = status
7
8     def info(self):
9         print(f"Mobil {self.merk} tahun {self.tahun} dengan harga sewa per hari {self.harga_sewa}")
10
11     def sewa(self):
12         if self.status == "tersedia":
13             print("Mobil berhasil disewa!")
14             self.status = "tidak tersedia"
15         else:
16             print("Maaf, mobil sedang tidak tersedia untuk disewa.")
17
18     def kembali(self):
19         if self.status == "tidak tersedia":
20             print("Terima kasih telah menyewa mobil kami!")
21             self.status = "tersedia"
22         else:
23             print("Maaf, mobil belum disewa atau sudah dikembalikan.")
```

```
44 mobil1 = Mobil("Honda Jazz", 2020, 300000, "tersedia")
45 mobil2 = MobilSUV("Toyota Fortuner", 2019, 500000, "tersedia", 7)
46 mobil3 = MobilPickup("Mitsubishi Triton", 2018, 700000, "tidak tersedia", 5, 1000)
47
48 mobil1.info()
49 mobil1.sewa()
50 mobil1.info()
51 mobil1.kembali()
52 mobil1.kembali()
53
54 mobil2.info()
55 mobil2.sewa()
56 mobil2.kembali()
57
58 mobil3.info()
59 mobil3.sewa()
60 mobil3.kembali()
61 mobil3.sewa()
```

```
25 class MobilSUV(Mobil):
26     def __init__(self, merk, tahun, harga_sewa, status, kapasitas):
27         super().__init__(merk, tahun, harga_sewa, status)
28         self.kapasitas = kapasitas
29
30     def info(self):
31         print(f"Meobil SUV {self.merk} tahun {self.tahun} dengan kapasitas {self.kapasitas} penumpang dan ")
32         print(f"harga sewa per hari {self.harga_sewa}")
```

```
34 class MobilPickup(Mobil):
35     def __init__(self, merk, tahun, harga_sewa, status, kapasitas, muatan_maksimal):
36         super().__init__(merk, tahun, harga_sewa, status)
37         self.kapasitas = kapasitas
38         self.muatan_maksimal = muatan_maksimal
39
40     def info(self):
41         print(f"Mobil pickup {self.merk} tahun {self.tahun} dengan kapasitas {self.kapasitas} penumpang dan ")
42         print(f"muatan maksimal {self.muatan_maksimal} kg, dan harga sewa per hari {self.harga_sewa}");
```

Magic Method *in* Python

Apa itu `__str__` dalam Python?

`__str__` merupakan salah satu dari beberapa magic method yang dipakai untuk mengatur bagaimana objek ditampilkan sebagai string. Jadi, kalau kita memanggil fungsi seperti `print()` atau `str()` terhadap sebuah objek, Python otomatis akan memanggil method `__str__()` dari objek tersebut.

1. Magic Methods untuk Representasi Objek (Object Representation)

Method	Fungsi
<code>__str__()</code>	Mengatur representasi string untuk user (friendly)
<code>__repr__()</code>	Mengatur representasi string teknis (untuk developer)
<code>__format__()</code>	Format custom saat pakai <code>format()</code> atau <code>f-string</code>
<code>__bytes__()</code>	Mengubah object jadi bytes

2. Magic Methods untuk Inisialisasi dan Destruksi (Initialization & Destruction)

Method	Fungsi
<code>__init__()</code>	Constructor (dipanggil saat objek dibuat)
<code>__del__()</code>	Destructor (dipanggil saat objek dihapus)

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

